Automatic Video Summarization using AI

VideoSummarize — Project Presentation

Methodology • System Architecture • Technical Tools • Conclusion

College Project — 2026

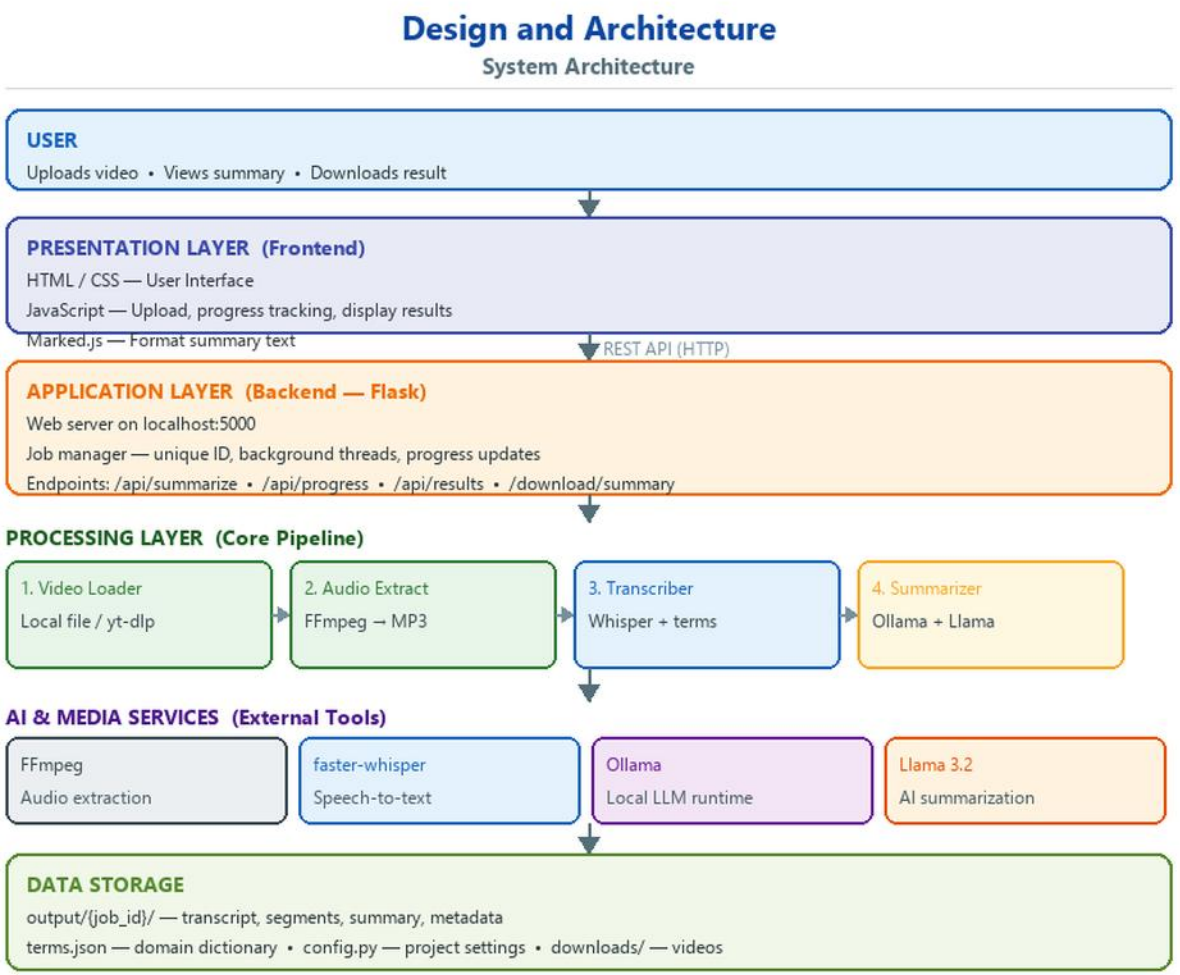
Design and Methodology

Methodology Flowchart



Design and Architecture

System Architecture



All processing runs locally — no cloud API, data stays on your machine

Implementation

Technical Tools

Core Stack

Python 3.9+
Backend language & pipeline logic

Flask 2.3+
Web server & REST API

HTML / CSS / JavaScript
Frontend user interface

Marked.js
Render formatted summary text

AI & Speech

faster-whisper
Speech-to-text (2–4× faster than Whisper)

Whisper base model
Local transcription (~140 MB)

Ollama
Local LLM runtime (no cloud API)

Llama 3.2 : 3b
AI summarization model

Media & Data

FFmpeg
Extract audio from video (MP4 → MP3)

yt-dlp
YouTube video download (CLI pipeline)

pydub
Audio processing support

terms.json
Domain-specific term correction

Supporting

threading
Background job processing

uuid
Unique job ID generation

Edge TTS / gTTS
Text-to-speech (extended pipeline)

config.py
Central project configuration

All tools run locally on the user's machine — privacy-first, no paid API keys required

Conclusion & Next Steps

Conclusion

- Built an end-to-end Automatic Video Summarization system using local AI.
- Converts long educational videos into concise, readable summaries.
- Pipeline: Upload → Extract Audio → Transcribe → Summarize → Display.
- Uses faster-whisper for speech-to-text and Ollama/Llama for summarization.
- Domain term correction improves accuracy for technical/educational content.
- Web UI with real-time progress tracking — fully functional and demo-ready.
- Privacy-first design: all processing runs locally, no cloud dependency.

Next Steps

- **YouTube URL in Web UI**
  Enable direct YouTube link input from the browser (CLI already supports yt-dlp).
- **TTS Audio Summary**
  Generate spoken summary using Edge TTS for audio learning.
- **SRT Subtitles**
  Export synchronized subtitle files linked to summary timestamps.
- **Job History**
  Store and browse past summarization jobs in the web interface.
- **GPU Acceleration**
  Use CUDA for faster Whisper transcription on supported hardware.
- **Multi-language Support**
  Extend transcription beyond English for broader use.
- **Mobile-friendly UI**
  Responsive design for phones and tablets.